



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
Disciplina: ALGORITMO E ESTRUTURA DE DADOS I

Semestre: 2026.2

Data: ____/____/____
PROFESSORA: Rosana Rego

— Exercício 7: Unidade I —

Complexidade de algoritmos: busca e ordenação

Nome: _____

Nota: _____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade7.c`, evolução da `atividade6.c`, cujo objetivo é demonstrar, de forma visual e interativa (usando a biblioteca **raylib**), os seguintes conceitos de linguagem C:

- **Complexidade de algoritmos:** medir o custo de um algoritmo contando operações (comparações, trocas) em vez de apenas “rodar e ver se é rápido”;
- **Ordenação** com *bubble sort*, de complexidade $O(n^2)$ no pior caso;
- **Busca sequencial**, de complexidade $O(n)$, que funciona em qualquer vetor;
- **Busca binária**, de complexidade $O(\log n)$, que só funciona corretamente em um vetor ordenado;
- Revisão: ponteiros, alocação dinâmica e *struct* (**Placar**).

O programa exibe um vetor de placares como barras verticais (altura proporcional à pontuação), permitindo enxergar visualmente o efeito da ordenação e comparar o número de comparações de cada algoritmo.

1 Estrutura de dados: o placar

Cada placar é uma *struct* simples, com um nome e uma pontuação, guardada em um vetor dinâmico:

```
1 typedef struct {
2     char nome[16];
3     int pontuacao;
4 } Placar;
5
6 Placar *criarPlacares(int quantidade) {
7     Placar *placares = (Placar *)malloc(quantidade * sizeof(Placar));
8     for (int i = 0; i < quantidade; i++) {
9         Placar *p = (placares + i);
10        TextCopy(p->nome, TextFormat("J%02d", i + 1));
11        p->pontuacao = GetRandomValue(10, PONTUACAO_MAX);
12    }
13    return placares;
14 }
```

2 Ordenação: bubble sort contando o custo

O *bubble sort* percorre o vetor repetidamente, trocando elementos vizinhos fora de ordem. A função abaixo conta cada **comparação** e cada **troca** realizada, para tornar visível o custo do algoritmo:

```
1 void ordenarBubbleSort(Placar *vetor, int n, long *comparacoes, long
   *trocas) {
2     *comparacoes = 0;
3     *trocas = 0;
4
5     for (int i = 0; i < n - 1; i++) {
6         for (int j = 0; j < n - 1 - i; j++) {
7             (*comparacoes)++;
8             if (vetor[j].pontuacao > vetor[j + 1].pontuacao) {
9                 Placar temp = vetor[j];
10                vetor[j] = vetor[j + 1];
11                vetor[j + 1] = temp;
12                (*trocas)++;
13            }
14        }
15    }
16 }
```

No pior caso (vetor em ordem decrescente), os dois laços aninhados executam aproximadamente n^2 comparações – por isso dizemos que o *bubble sort* é $O(n^2)$.

3 Busca sequencial: $O(n)$

A busca sequencial verifica cada posição do vetor, uma por uma, até encontrar o valor procurado (ou chegar ao fim). Ela funciona em **qualquer** vetor, ordenado ou não:

```
1 int buscaSequencial(Placar *vetor, int n, int alvo, long *comparacoes
   ) {
2     *comparacoes = 0;
3     for (int i = 0; i < n; i++) {
4         (*comparacoes)++;
5         if (vetor[i].pontuacao == alvo) return i;
6     }
7     return -1;
8 }
```

4 Busca binária: $O(\log n)$

A busca binária é muito mais rápida, mas exige que o vetor esteja **ordenado**: a cada passo, ela descarta metade dos elementos restantes comparando o alvo com o elemento do meio:

```
1 int buscaBinaria(Placar *vetor, int n, int alvo, long *comparacoes) {
2     *comparacoes = 0;
3     int inicio = 0, fim = n - 1;
4
5     while (inicio <= fim) {
6         (*comparacoes)++;
7         int meio = (inicio + fim) / 2;
8
9         if (vetor[meio].pontuacao == alvo) return meio;
```

```

10         if (vetor[meio].pontuacao < alvo) inicio = meio + 1;
11         else fim = meio - 1;
12     }
13     return -1;
14 }

```

Por isso, no programa, a tecla W (busca binária) só é aceita se `ordenado == true`; caso contrário, o resultado da busca poderia estar incorreto.

Complexidade dos algoritmos desta atividade

Algoritmo	Complexidade (pior caso)	Pré-requisito
ordenarBubbleSort	$O(n^2)$	nenhum
buscaSequencial	$O(n)$	nenhum
buscaBinaria	$O(\log n)$	vetor ordenado

Quanto maior o vetor, maior a vantagem da busca binária: para $n = 1000$, a busca sequencial pode exigir até 1000 comparações, enquanto a busca binária exige, no máximo, cerca de $\log_2 1000 \approx 10$.

5 Exercícios

Exercício 1 — Insertion sort

Atualmente o único algoritmo de ordenação disponível é o *bubble sort*.

Tarefa: implemente `void ordenarInsertionSort(Placar *vetor, int n, long *comparacoes, long *trocas)` e associe-o a uma nova tecla (I).

Dicas:

- No *insertion sort*, cada elemento é comparado com os anteriores e “inserido” na posição correta, deslocando os demais.
- Conte uma comparação a cada teste `vetor[j].pontuacao > chave.pontuacao` dentro do laço interno.
- Compare, na tela, o número de comparações do *insertion sort* com o do *bubble sort* para o mesmo vetor.

Exercício 2 — Medindo tempo real de execução

Atualmente só medimos o custo em número de comparações, não em tempo real.

Tarefa: meça, em milissegundos, quanto tempo `ordenarBubbleSort` leva para executar, usando `GetTime()` do raylib antes e depois da chamada.

Dicas:

- Guarde `double inicio = GetTime();` antes de chamar a função e `double fim = GetTime();` depois.
- O tempo decorrido, em milissegundos, é `(fim - inicio) * 1000.0`.
- Exiba esse valor com `DrawText` e `TextFormat`, junto com o número de comparações.